

Overview

This project is based on a real-world FinTech task drawn from the GDPval Task Bank, set in the context of equity capital markets. The scenario assumes the date is April 11, 2025, and our group acts as Investment Banking Analysts within an Equity Capital Markets (ECM) group. Our goal is to generate a comprehensive analysis of the S&P 500 index to understand where Price-to-Earnings (P/E) multiples stand across all companies and their respective sub-sectors.

The S&P 500's historical average P/E typically ranges between 15- and 20-times earnings. We were required to find which sub-sectors and individual companies are trading above or below this historical benchmark, which signals potential overvaluation or undervaluation in the current market. The deliverable was a structured dataset covering all 503 S&P 500 companies, containing the last twelve months (LTM) P/E and next twelve months (NTM), dividend yield, annual and quarterly EPS estimate, market capitalisation and percentage weight in index.

AI-Augmented Pipeline and Deliverable

Data was sourced from Finviz, a publicly available stock screening platform. We used the finvizlite Python library which wraps Finviz's web screener and handles HTTP requests and HTML parsing internally, the LLM identified this library after early iterations with direct web scraping proved slow and brittle. The screener was filtered to S&P 500 members (idx_sp500) and configured to retrieve eleven fields: ticker, company, sector, industry, market cap, trailing P/E, forward P/E, dividend yields, EPS and next quarter EPS estimate which returned 503 companies. Market capitalisation values were parsed to billions and a percentage-of-index column was derived by dividing each company's market cap by the total index cap of \$64,802 billion. The cleaned dataset was then formatted into a structured Excel workbook. Each cell of code was manually reviewed before execution and outputs were verified. For example, we verified the \$64.8T market cap against published benchmarks. A standardise () function maps Finviz's raw HTML column names, parses market cap strings like "2.85T" into numeric millions, strips percentage signs and changes all values to floats. Negative or zero P/E ratios are set to null so they don't affect the aggregate.

The final Excel book contains four sheets. The Master Table lists all 503 companies with the fields listed above. Each company's NTM P/E cell is conditionally formatted, red for above average, green for below and yellow for within. This means a reader can scan the column visually without reading every number. The Sub-Sector Summary and Sector Summary sheets aggregate these values at the GICS sub-industry and sector level respectively, showing weighted-average P/E, weighted dividend yield, median P/E, company count, and total market cap for each grouping. Both include auto-filters so a client can sort by any column immediately. An Extremes Dashboard surfaces the 25 most expensive and 25 cheapest companies and sub-sectors by NTM P/E in separate tables which gives a direct answer to the core question.

An LLM was used at two stages. First, the LLM was prompted to identify the most appropriate Python library for retrieving Finviz data without requiring an API key. Early iterations used yfinance for data acquisition, however, yfinance rate-limits aggressively and returns incomplete forward-EPS coverage. Through iterative prompting, we pivoted to a Finviz-scraping approach, which returned all 503 companies in under 60 seconds with better field coverage.

Evaluation Rubric

We assessed the deliverable across five dimensions:

- **Correctness:** We spot checked 15 companies against Bloomberg values. LTM P/E matched with an outlier of Tesla due to Finviz use of a slightly different trailing-earnings window.
- **Completeness:** 503 of 503 S&P 500 companies were captured. Gaps are due to lack of analyst consensus or negative trailing earnings rather than data quality. They are to be expected.
- **Clarity:** Headers use industry standard terminology. Color coding adequately maps pricing ranges (expensive, cheap, in-range). Someone without technical experience should be able to navigate the document without instruction.
- **Format:** The sheet follows formatting appropriate for the desired output including navy headers and auto-filters.
- **Usefulness:** The document directly answers the task at hand, providing which sectors and companies are trading above or below historical average.

Reflection and Risk Assessment

What worked:

The finvizlite library took away all HTTP handling and pagination, returning the full index in a single function call. This was a significant improvement over the earlier yfinance approach which would have taken 10-15 minutes and still returned incomplete data.

What struggled:

Finviz doesn't provide a true calendar year + 1 annual EPS field. The EPS column is trailing twelve months and the only forward figure is next quarter. If used to an industry level, an analyst would source data from elsewhere that Finviz can't provide. A real ECM analyst would cross-reference the output against a Bloomberg terminal screen and adjust discrepancies within the data rather than using consensus from a free screener. Finviz uses its own industry labels that don't map perfectly to GICS sub-industry codes which means that certain groups (Banks – Regional) won't map directly to what a client would see on a Bloomberg terminal.

Ethical and data-quality risks:

Finviz restricts automated scraping for commercial redistribution so real engagement would require a licensed data feed. The pipeline doesn't carry traditional LLM hallucination risk as the figures come directly from the screener not from LLM generation. However, AI-generated formatting code could introduce new errors such as misaligned column reference that are difficult to catch in a 500-row spreadsheet. This was mitigated by printing a console summary with weighted-average P/E and total market cap (\$64.8T) and confirming these against published S&P 500 benchmarks.

A risk to data-quality is that there is no retry logic per ticker. If Finviz returns a stale or missing value for one company (ticker halted or quote delayed) you'd have no way of knowing from just the output.